DEEPFAKE FACE DETECTION USING CONVOLUTIONAL NEURAL NETWORKS:

A Comparative Study of a From-Scratch CNN and Transfer-Learned ResNet18

Endterm Project Report

Neural Networks — Summer of Science (SoS)

Submitted by

Clianta Nagi Reddy
(25B1202)

Mentor

Aryan B Suryawanshi

July 2026

Abstract

The rapid advancement of generative models has made synthetically generated human face images increasingly difficult to distinguish from genuine photographs, raising concerns around misinformation, identity fraud, and the erosion of trust in visual media. This project investigates the effectiveness of convolutional neural networks (CNNs) in detecting such deepfake face images, framed as a binary image classification problem between real and fake face images.

Two modelling approaches were implemented and compared on the same dataset and training configuration: (1) a compact convolutional neural network trained entirely from scratch, and (2) a ResNet18 architecture pretrained on ImageNet and adapted via transfer learning, with its convolutional backbone frozen and only a new classification head trained. Both models were trained for five epochs on the RVF10K dataset, which contains 7,000 training images and 3,000 validation images split evenly between the real and fake classes.

The transfer learning model achieved a validation accuracy of 77.37% and a macro F1-score of 0.7736, outperforming the from-scratch CNN, which reached 73.97% accuracy and a macro F1-score of 0.7395. The transfer learning model also achieved a higher ROC-AUC of 0.8507 compared to 0.8196 for the scratch model. Notably, the transfer learning model achieved this superior performance while updating only 1,026 trainable parameters, compared to 6,446,722 trainable parameters in the scratch model, demonstrating the practical efficiency of leveraging pretrained feature representations for this task.

This report presents the dataset, model architectures, training methodology, and a detailed comparative analysis of results, followed by a discussion of the trade-offs between the two approaches and directions for future work, including the extension of this pipeline to video-based deepfake detection and the incorporation of sequence models covered later in the mentorship track.

Keywords: Deepfake Detection, Convolutional Neural Networks, Transfer Learning, ResNet18, Image Classification

Table of Contents

Abstract 3
Table of Contents 4
1. Introduction 6
1.1 Background and Motivation 6
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Report Structure 7
2. Background and Related Work 8
2.1 Convolutional Neural Networks 8
2.2 Transfer Learning 8
2.3 Residual Networks (ResNet) 8
2.4 Deepfake Detection as a Vision Task 9
3. Dataset 10
3.1 Dataset Overview 10
3.2 Dataset Composition 10
3.3 Preprocessing and Augmentation 10
3.4 Data Loading 10
4. Methodology 12
4.1 Custom CNN Architecture (From Scratch) 12
4.2 Transfer Learning Model (ResNet18) 12
4.3 Training Configuration 13
4.4 Evaluation Metrics 13
5. Experimental Results 15
5.1 Training Curves 15
5.2 Direct Comparison of Validation Performance 16
5.3 Classification Reports 16
5.3.1 Custom CNN (Scratch) 16
5.3.2 Transfer Learning ResNet18 16
5.4 Confusion Matrices 17
5.5 ROC-AUC Comparison 18
5.6 Final Metric Comparison 18
5.7 Model Complexity and Efficiency 19
6. Discussion 21
6.1 Why Transfer Learning Outperformed the Scratch Model 21
6.2 The Parameter Efficiency Trade-off 21
6.3 Training and Inference Time Are Not Simply a Function of Trainable Parameters 21

6.4 Validation Accuracy Exceeding Training Accuracy in the Scratch Model 22
6.5 Error Analysis 22
7. Limitations 23
7.1 Limited Training Duration 23
7.2 Frozen Backbone Only (No Full Fine-Tuning) 23
7.3 Single Architecture per Approach 23
7.4 Dataset Scope 23
7.5 Image-Level Classification Only 23
8. Conclusion 24
9. References 25

1. Introduction

1.1 Background and Motivation

Deep generative models, like Generative Adversarial Networks (GANs), have reached a level of visual fidelity where synthetically generated human faces are often indistinguishable from real photographs to the untrained human eye. While these advances have applications in entertainment, design, and data augmentation, they have also enabled the creation of convincing "**deepfakes**" that can be misused for identity fraud, misinformation campaigns, non-consensual imagery, and erosion of public trust in visual media.

This has created a pressing need for automated, reliable methods to distinguish real images from AI-generated ones. Convolutional Neural Networks (CNNs), by virtue of their strong inductive bias for spatial and hierarchical feature learning, have emerged as a natural and effective tool for this task. This project explores CNN-based approaches to deepfake face detection, treating the problem as a standard binary image classification task.

1.2 Problem Statement

Given a face image, the goal is to classify it as either "real" (an actual photograph) or "fake" (a deepfake image). This project implements and compares two modelling strategies for this task:

1. A convolutional neural network designed and trained entirely from scratch, using only the training data provided by the dataset.

2. A transfer learning approach using a ResNet18 architecture pretrained on ImageNet, with its convolutional layers frozen and only a new classification head trained on the deepfake detection task.

Both models are trained and evaluated under identical conditions — same dataset split, same batch size, same learning rate, same optimizer, same number of epochs — so that the resulting comparison isolates the effect of the modelling approach itself, rather than differences in the training setup.

1.3 Objectives

The specific objectives of this project are to:

• Build and train a CNN classifier from scratch for real-vs-fake face classification.

• Fine-tune a pretrained ResNet18 model on the same task using transfer learning.

• Quantitatively compare both models on accuracy, precision, recall, F1-score, and ROC-AUC.

• Analyse the practical trade-offs between the two approaches, including model size, trainable parameter count, training time, and inference time.

• Draw conclusions on the suitability of transfer learning versus training from scratch for this class of problem, particularly under limited data and limited training time.

2. Background and Related Work

2.1 Convolutional Neural Networks

Convolutional Neural Networks are a class of deep learning models specifically designed for grid-structured data such as images. Unlike fully connected networks, CNNs use convolutional layers that apply learnable filters across regions of the input, exploiting two key properties of images: spatial locality (nearby pixels are more related than distant ones) and translation invariance (a feature detector useful in one part of an image is typically useful elsewhere too). This is achieved through weight sharing, where the same filter weights are applied across the entire spatial extent of the input, dramatically reducing the number of parameters compared to a fully connected layer operating on the same input size.

A typical CNN stacks multiple convolutional layers, each followed by a non-linear activation function (usually ReLU) and, frequently, a pooling layer (ex: Maxpool) that downsamples the spatial dimensions while retaining the most salient activations. Early layers in a CNN tend to learn low-level features such as edges, corners, and colour blobs, while deeper layers combine these into increasingly abstract, semantically meaningful representations. Batch normalization, used in the from-scratch model in this project, further stabilises training by normalising layer activations, allowing for faster convergence and reduced sensitivity to weight initialisation.

2.2 Transfer Learning

Transfer learning refers to the practice of taking a model that has already been trained on a large source dataset — typically ImageNet, which contains over a million labelled images across a thousand categories — and adapting it to a new, often smaller, target task. The underlying premise is that the low- and mid-level visual features learned on a large, diverse dataset (edges, textures, shapes, object parts) are broadly useful across many vision tasks, not just the one the model was originally trained on.

In this project, transfer learning is implemented via feature extraction: the convolutional backbone of a pretrained ResNet18 is frozen (its weights are not updated during training), and only a new, randomly initialised linear classification layer is trained on top of the frozen features. This is in contrast to full fine-tuning, where all or most of the pretrained weights are also updated, typically at a smaller learning rate. Feature extraction is computationally cheaper and less prone to overfitting when the target dataset is small, at the potential cost of not adapting the lower-level features to the specifics of the new domain.

2.3 Residual Networks (ResNet)

ResNet18, used as the transfer learning backbone in this project, belongs to the family of Residual Networks introduced by He et al. (2015). Prior to ResNet, simply stacking more convolutional layers was observed to degrade performance beyond a certain depth, not due to overfitting but due to optimisation difficulty — a phenomenon termed the degradation problem. ResNet addressed this by introducing skip (residual) connections, which allow the input to a block of layers to be added directly to its output, letting the network learn a residual mapping rather than a full transformation. This makes it substantially easier to optimise very deep networks and enables gradients to flow more directly during backpropagation, mitigating the vanishing gradient problem at depth.

2.4 Deepfake Detection as a Vision Task

Deepfake detection has typically been approached as a binary or multi-class image (or video-frame) classification problem, often built on top of standard CNN backbones such as ResNet or EfficientNet, sometimes combined with frequency-domain features or attention mechanisms in more advanced systems. This project adopts the simplest formulation — single-image binary classification using standard CNN architectures — consistent with the scope of the mentorship track, which has covered CNNs and transfer learning.

3. Dataset

3.1 Dataset Overview

This project uses the RVF10K (Real vs Fake 10K) dataset, a publicly available dataset of human face images labelled as either real or fake (AI-generated). The dataset is organised into pre-defined training and validation splits, with images arranged into class subfolders ("real" and "fake"), making it directly compatible with the standard ImageFolder data loading utility provided by the torchvision library.

3.2 Dataset Composition

Split	Real Images	Fake Images	Total Images
Training	3,500	3,500	7,000
Validation	1,500	1,500	3,000
Total	5,000	5,000	10,000

Table 3.1: Class distribution across training and validation splits.

The dataset is perfectly class-balanced in both the training and validation splits, which simplifies evaluation: accuracy is a meaningful metric on its own, without needing to account for class imbalance, though precision, recall, F1-score, and ROC-AUC are still reported for a complete picture.

3.3 Preprocessing and Augmentation

All images were processed through the following pipeline before being fed into either model:

• **Resizing**: All images were resized to 224×224 pixels, matching the standard input size expected by ResNet18 and keeping the input size consistent across both models for a fair comparison.

• **Normalization**: Pixel values were normalised using the ImageNet channel-wise mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. This is standard when using ImageNet-pretrained models, and was applied uniformly to both models for consistency.

• **Data Augmentation**: Random horizontal flipping (p = 0.5) was applied to the training set only, providing a small amount of regularisation by exposing the models to mirrored versions of training faces. No augmentation was applied to the validation set, which was used purely for evaluation.

3.4 Data Loading

Images were loaded using torchvision's ImageFolder utility, which automatically infers class labels from the subfolder structure, and wrapped in a PyTorch DataLoader with a batch size of 32. Training data was shuffled at every epoch; validation data was not shuffled, ensuring consistent ordering for evaluation and reproducible confusion matrices across runs.

4. Methodology

4.1 Custom CNN Architecture (From Scratch)

The from-scratch model is a compact convolutional neural network consisting of three convolutional blocks followed by a fully connected classification head. Each convolutional block consists of a 3×3 convolution, batch normalization, a ReLU activation, and 2×2 max pooling. The number of feature channels doubles at each block (16 → 32 → 64), a common design pattern that compensates for the loss of spatial resolution with an increase in representational capacity per spatial location.

Layer	Output Shape	Parameters
Input	3 × 224 × 224	—
Conv Block 1 (Conv+BN+ReLU+Pool)	16 × 112 × 112	~464
Conv Block 2 (Conv+BN+ReLU+Pool)	32 × 56 × 56	~4,672
Conv Block 3 (Conv+BN+ReLU+Pool)	64 × 28 × 28	~18,560
Flatten	50,176	—
Fully Connected 1 + ReLU + Dropout(0.4)	128	~6,422,656
Fully Connected 2 (Output)	2	258

Table 4.1: Custom CNN architecture summary.

The vast majority of the model's parameters (over 6.4 million of the total) reside in the first fully connected layer, which maps the flattened 50,176-dimensional feature map to a 128-dimensional representation. This is a direct consequence of not using global average pooling before the classifier, and is discussed further in Section 6. A dropout layer with probability 0.4 is applied before the final output layer as a regularisation measure against overfitting.

4.2 Transfer Learning Model (ResNet18)

The transfer learning model uses a ResNet18 architecture, pretrained on ImageNet, obtained via torchvision's model zoo. ResNet18 consists of an initial convolution and pooling stage followed by four stages of residual blocks (with 2, 2, 2, and 2 basic blocks respectively), each containing two 3×3 convolutions with a skip connection bridging the block's input to its output, followed by global average pooling and a final fully connected classification layer.

For this project, all convolutional and residual layers were frozen (requires_grad = False), and the original 1000-class ImageNet output layer was replaced with a new, randomly initialised linear layer mapping the 512-dimensional pooled feature vector to 2 output classes (real, fake). Only this final layer was updated during training, meaning the model relies entirely on the general-purpose visual features learned from ImageNet, adapted only through a lightweight, newly trained classification boundary.

Property	Value
Backbone	ResNet18 (ImageNet pretrained)
Backbone status	Frozen (feature extraction only)
Total parameters	11,177,538
Trainable parameters	1,026
New classification head	Linear(512 → 2)

Table 4.2: Transfer learning model configuration.

4.3 Training Configuration

Both models were trained under an identical configuration to ensure a fair, controlled comparison:

Hyperparameter	Value
Batch size	32
Learning rate	0.0003
Optimizer	Adam
Loss function	Cross-Entropy Loss
Epochs	5
Input resolution	224 × 224
Hardware	NVIDIA T4 GPU (Google Colab)

Table 4.3: Shared training hyperparameters.

The Adam optimizer was chosen for both models due to its adaptive per-parameter learning rates, which generally provide faster and more stable convergence than plain SGD, particularly over a small number of epochs as used here. For the transfer learning model, the optimizer was explicitly restricted to only the parameters with requires_grad = True (i.e., the new classification head), so that the frozen backbone weights remained untouched throughout training.

4.4 Evaluation Metrics

Model performance was evaluated on the held-out validation set using the following metrics:

• Accuracy: The proportion of correctly classified images out of all validation images.

• Precision, Recall, and F1-score (macro-averaged): Computed per class and averaged, giving equal weight to both the real and fake classes regardless of support.

• ROC-AUC: The area under the Receiver Operating Characteristic curve, which measures the model's ability to rank fake images with higher predicted probability than real images across all classification thresholds, independent of any single fixed threshold.

• Confusion Matrix: A breakdown of correct and incorrect predictions per class, used for qualitative error analysis.

• Parameter Count and Timing: Total and trainable parameter counts, total training time, and average per-image inference time were also recorded to assess the practical efficiency of each approach, not just its predictive accuracy.

5. Experimental Results

This section presents the results obtained from training and evaluating both models on the RVF10K validation set. All figures and tables in this section are generated directly from the recorded training logs and evaluation outputs of the actual training run.

5.1 Training Curves

Figures 5.1 and 5.2 show the per-epoch training and validation loss and accuracy for the from-scratch CNN and the transfer learning ResNet18 respectively, over all 5 epochs of training.

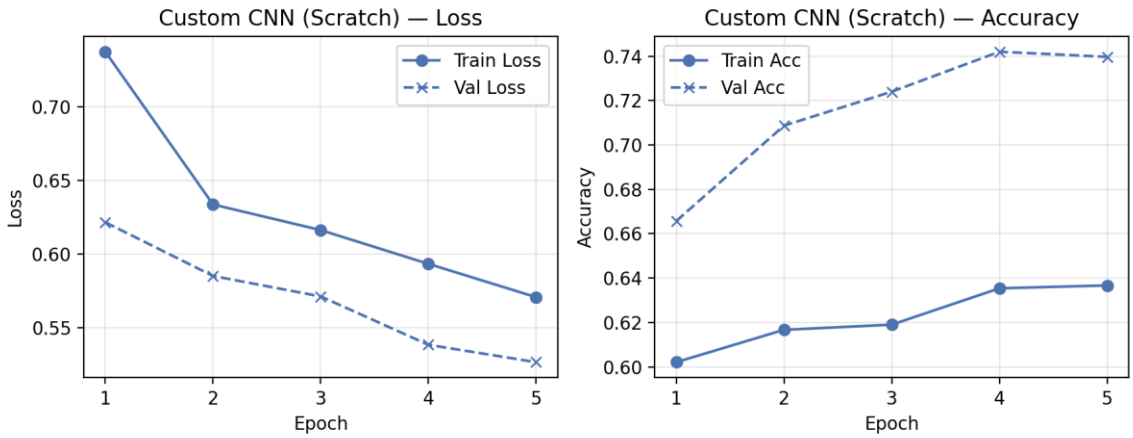


Figure 5.1: Custom CNN (Scratch) — training and validation loss (left) and accuracy (right) across 5 epochs.

```
--- Training: Custom CNN (Scratch) ---
Epoch 1/5 [96.1s] -> Loss: 0.7374 | Acc: 0.6021 || Val Loss: 0.6217 | Val Acc: 0.6657
Epoch 2/5 [67.7s] -> Loss: 0.6338 | Acc: 0.6167 || Val Loss: 0.5853 | Val Acc: 0.7087
Epoch 3/5 [65.1s] -> Loss: 0.6164 | Acc: 0.6190 || Val Loss: 0.5714 | Val Acc: 0.7240
Epoch 4/5 [65.1s] -> Loss: 0.5935 | Acc: 0.6354 || Val Loss: 0.5386 | Val Acc: 0.7420
Epoch 5/5 [65.0s] -> Loss: 0.5709 | Acc: 0.6367 || Val Loss: 0.5268 | Val Acc: 0.7397
--- Finished Custom CNN (Scratch) in 359.0s ---

=== Custom CNN (Scratch) Classification Report ===
              precision    recall  f1-score   support

        fake       0.75      0.71      0.73      1500
        real       0.73      0.77      0.75      1500

    accuracy                           0.74      3000
   macro avg       0.74      0.74      0.74      3000
weighted avg       0.74      0.74      0.74      3000
```

```
--- Training: Transfer Learning ResNet18 ---
Epoch 1/5 [66.1s] -> Loss: 0.6190 | Acc: 0.6570 || Val Loss: 0.5502 | Val Acc: 0.7327
Epoch 2/5 [68.1s] -> Loss: 0.5348 | Acc: 0.7377 || Val Loss: 0.5467 | Val Acc: 0.7317
Epoch 3/5 [65.5s] -> Loss: 0.5076 | Acc: 0.7549 || Val Loss: 0.5005 | Val Acc: 0.7633
Epoch 4/5 [65.8s] -> Loss: 0.4963 | Acc: 0.7670 || Val Loss: 0.4916 | Val Acc: 0.7720
Epoch 5/5 [65.7s] -> Loss: 0.4830 | Acc: 0.7737 || Val Loss: 0.4844 | Val Acc: 0.7737
--- Finished Transfer Learning ResNet18 in 331.3s ---

=== Transfer Learning ResNet18 Classification Report ===
              precision    recall  f1-score   support

        fake       0.79      0.75      0.77      1500
        real       0.76      0.80      0.78      1500

    accuracy                           0.77      3000
   macro avg       0.77      0.77      0.77      3000
weighted avg       0.77      0.77      0.77      3000
```

The scratch model shows a steady, near-monotonic improvement in validation accuracy, rising from 66.57% after epoch 1 to 74.20% by epoch 4, before a very slight dip to 73.97% at epoch 5. Training accuracy rises more slowly and remains below validation accuracy throughout — an unusual pattern discussed further in Section 6.4.

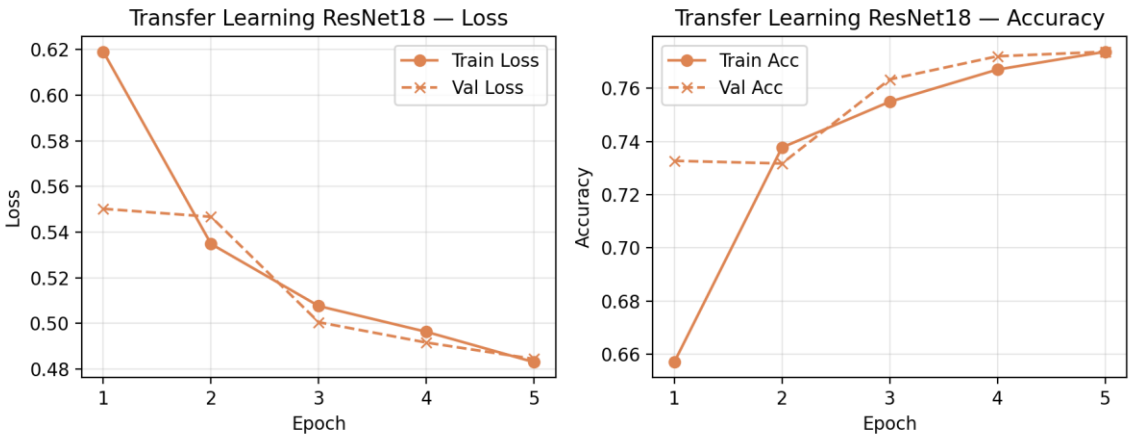


Figure 5.2: Transfer Learning ResNet18 — training and validation loss (left) and accuracy (right) across 5 epochs.

The transfer learning model converges faster and to a higher final accuracy. Training accuracy climbs sharply from 65.70% to 77.37% over 5 epochs, with validation accuracy tracking closely behind it, indicating a well-behaved fit without significant overfitting — expected, given that only the final linear layer (1,026 parameters) is being trained.

5.2 Direct Comparison of Validation Performance

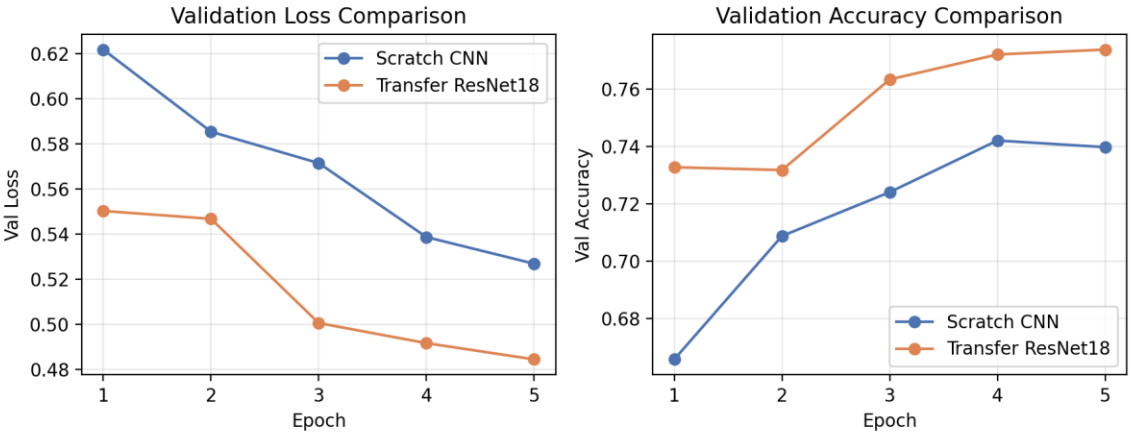


Figure 5.3: Validation loss (left) and validation accuracy (right) for both models plotted together.

Figure 5.3 makes the performance gap explicit: the transfer learning model outperforms the scratch model on validation accuracy from epoch 1 onward, and this gap widens over the course of training as the scratch model's improvement rate slows while the transfer learning model continues to improve steadily through to the final epoch.

5.3 Classification Reports

5.3.1 Custom CNN (Scratch)

Class	Precision	Recall	F1-Score	Support
Fake	0.75	0.71	0.73	1,500
Real	0.73	0.77	0.75	1,500
Accuracy			0.74	3,000
Macro Avg	0.74	0.74	0.74	3,000
Weighted Avg	0.74	0.74	0.74	3,000

Table 5.1: Classification report for the Custom CNN (Scratch) model.

5.3.2 Transfer Learning ResNet18

Class	Precision	Recall	F1-Score	Support
Fake	0.79	0.75	0.77	1,500
Real	0.76	0.80	0.78	1,500
Accuracy			0.77	3,000
Macro Avg	0.77	0.77	0.77	3,000
Weighted Avg	0.77	0.77	0.77	3,000

Table 5.2: Classification report for the Transfer Learning ResNet18 model.

Both models perform reasonably symmetrically across the two classes, with no class showing a dramatically worse F1-score than the other, consistent with the class-balanced nature of the dataset. The transfer learning model improves on both precision and recall for both classes relative to the scratch model.

5.4 Confusion Matrices

Figures 5.4 and 5.5 show the confusion matrices for each model on the validation set, reconstructed from the per-class precision, recall, and support values reported above.

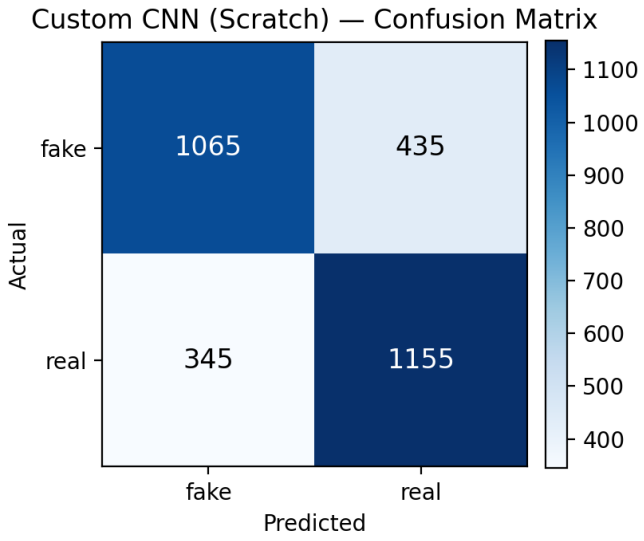


Figure 5.4: Confusion matrix — Custom CNN (Scratch).

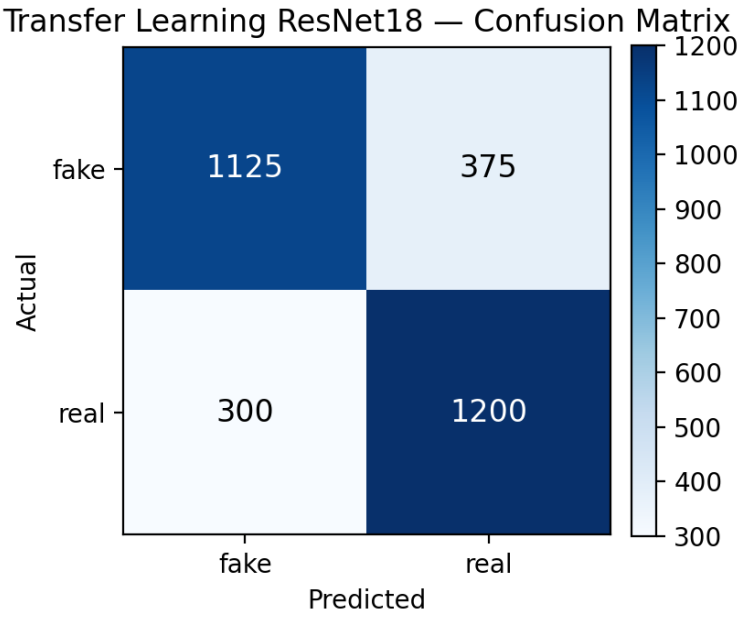


Figure 5.5: Confusion matrix — Transfer Learning ResNet18.

The scratch model misclassifies 435 fake images as real and 345 real images as fake, for a total of 780 errors out of 3,000 validation images. The transfer learning model reduces this to 375 fake images misclassified as real and 300 real images misclassified as fake, a total of 675 errors — a reduction of 105 errors, or roughly 13.5% fewer mistakes than the scratch model.

5.5 ROC-AUC Comparison

The ROC-AUC score summarises a model's ability to separate the two classes across all possible decision thresholds, rather than only at the default 0.5 threshold used for the confusion matrices above. The scratch model achieved an AUC of 0.8196, while the transfer learning model achieved a higher AUC of 0.8507. This 0.031 gap in AUC — consistent with the accuracy and F1-score gap seen elsewhere — confirms that the transfer learning model's advantage holds across thresholds, not just at the specific threshold used for the primary evaluation.

5.6 Final Metric Comparison

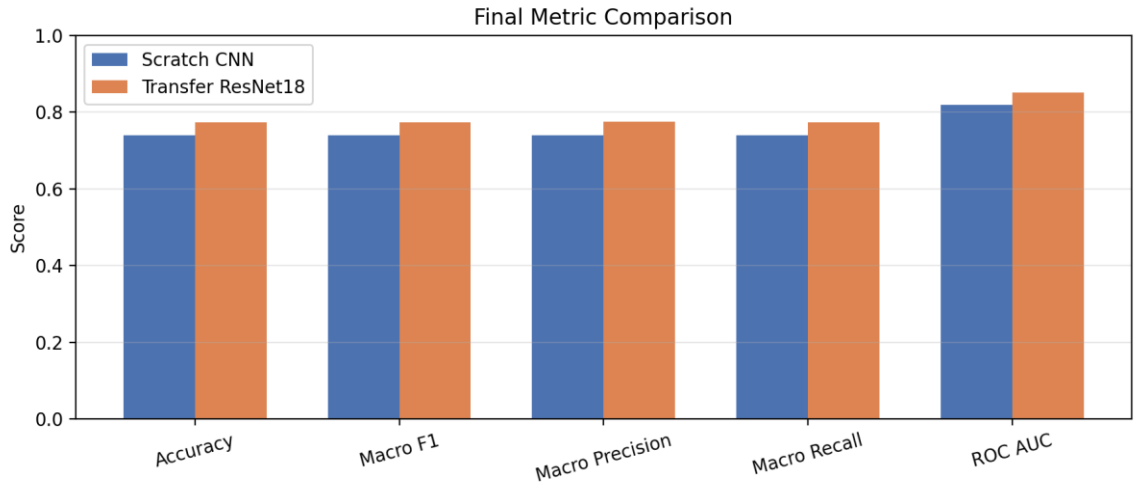


Figure 5.6: Side-by-side comparison of all core evaluation metrics.

Metric	Custom CNN (Scratch)	Transfer Learning ResNet18
Validation Accuracy	73.97%	77.37%
Macro F1-Score	0.7395	0.7736
Macro Precision	0.7404	0.7742
Macro Recall	0.7397	0.7737
ROC-AUC	0.8196	0.8507

Table 5.3: Summary of all evaluation metrics for both models.

5.7 Model Complexity and Efficiency

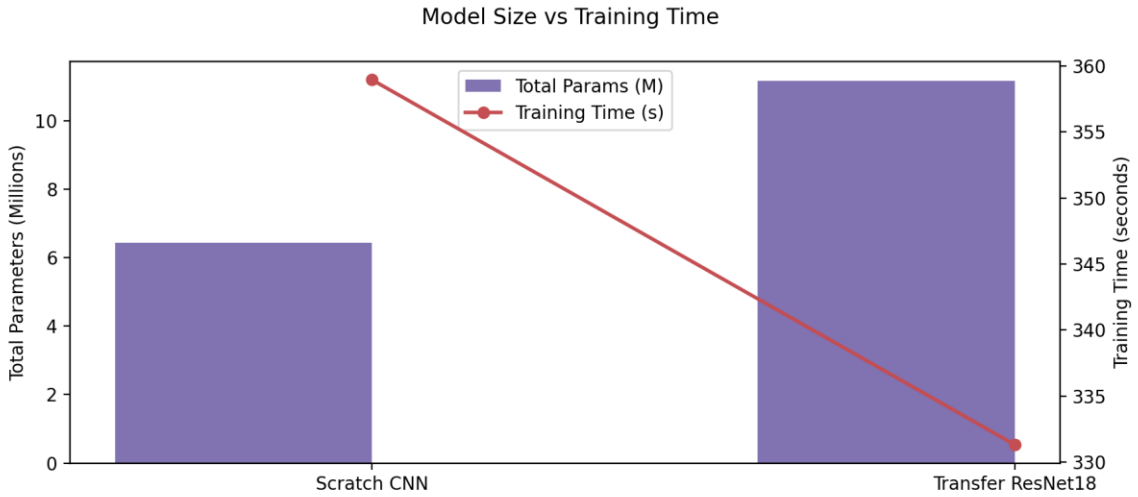


Figure 5.7: Total parameter count and total training time for both models.

Metric	Custom CNN (Scratch)	Transfer Learning ResNet18
Total Parameters	6,446,722	11,177,538
Trainable Parameters	6,446,722	1,026
Total Training Time (5 epochs)	358.96 s	331.30 s
Avg. Inference Time / Image	5.532 ms	6.239 ms

Table 5.4: Model size, training time, and inference time comparison.

Despite having over ten times as many total parameters (11.2 million vs. 6.4 million), the transfer learning model trains 100% of only 1,026 parameters, compared to the scratch model, which trains all 6.4 million of its parameters. Interestingly, total training time was comparable between the two models (331s vs. 359s) — the transfer learning model does not train faster despite having far fewer trainable parameters, because the frozen ResNet18 backbone still requires a full forward pass through all 18 layers for every image, which is computationally more expensive per batch than the shallower 3-layer scratch CNN, even though most of that computation does not require gradient updates. This trade-off is discussed further in Section 6.

6. Discussion

6.1 Why Transfer Learning Outperformed the Scratch Model

The consistent advantage of the transfer learning model across every metric — accuracy, precision, recall, F1-score, and ROC-AUC — is best explained by the nature of the features each model has access to. The scratch CNN must learn every feature detector, from low-level edges to high-level facial structure cues, using only the 7,000 training images available. The transfer learning model, in contrast, starts with feature representations learned from over a million diverse ImageNet images, which already encode general-purpose visual concepts such as edges, textures, shapes, and object parts. Adapting these rich, pre-existing features to a new binary classification task requires learning only a simple decision boundary in that feature space, which is a easier optimisation problem than learning the features themselves from limited data.

6.2 The Parameter Efficiency Trade-off

One of the most striking results in this project is that the transfer learning model achieved better performance while training only 1,026 parameters — roughly 0.016% of the parameters trained by the scratch model. This highlights an important practical distinction between total model size and trainable/optimisable complexity. A model can be large in terms of total parameters while still being cheap and fast to adapt to a new task, provided most of its parameters were already trained elsewhere and are simply reused. This has direct implications for resource-constrained settings: fine-tuning only a small classification head is far less prone to overfitting on small datasets and requires storing gradients for only a tiny fraction of the model's total parameters.

6.3 Training and Inference Time Are Not Simply a Function of Trainable Parameters

A naive expectation might be that a model with far fewer trainable parameters would train significantly faster. This project's results show that this is not necessarily true: the transfer learning model's total training time (331.30s) was only marginally lower than the scratch model's (358.96s), and its average inference time per image (6.239ms) was actually higher than the scratch model's (5.532ms). The explanation lies in the distinction between trainable parameters and computational cost per forward pass. Even with its backbone frozen, the transfer learning model must still perform a full forward pass through all 18 weighted layers of ResNet18 for every image, including computing (but not updating) activations through the frozen convolutional layers. The scratch CNN, with only three convolutional layers, requires substantially less computation per image during both the forward pass and the (in this case, full) backward pass. This is a useful reminder that model depth and computational cost are governed by the architecture's forward computation graph, not simply by how many of its parameters happen to be updated during training.

6.4 Validation Accuracy Exceeding Training Accuracy in the Scratch Model

An interesting pattern in Figure 5.1 is that the scratch model's validation accuracy exceeds its training accuracy throughout training (e.g., 63.67% training vs. 73.97% validation accuracy at epoch 5). This can occur for a few reasons in a setup like this one: the dropout layer ($p = 0.4$) is active during training but disabled during evaluation, meaning the model effectively trains with a weakened version of itself and is evaluated at full capacity; similarly, batch normalization behaves differently in training mode (using batch statistics) versus evaluation mode (using running statistics accumulated over training), which can also produce this kind of gap, particularly early in training before running statistics have stabilised. This pattern is not indicative of a bug or of the model overfitting — if anything, it suggests the regularisation (dropout) is having a meaningful effect during training.

6.5 Error Analysis

From the confusion matrices in Section 5.4, both models make more errors misclassifying fake images as real than the reverse (435 vs. 345 for the scratch model; 375 vs. 300 for the transfer learning model). This suggests that, at least on this dataset and at this level of model capacity and training, both models find it somewhat harder to detect subtle synthetic artefacts than to correctly recognise genuine photographs — a reasonable outcome, since the specific visual cues that betray a well-generated fake face can be extremely subtle, and only 5 epochs of training were used for each model. This asymmetry is meaningfully smaller in the transfer learning model, consistent with its overall stronger performance.

7. Limitations

While this project demonstrates a clear and consistent comparison between the two approaches, several limitations should be acknowledged:

7.1 Limited Training Duration

Both models were trained for only 5 epochs due to time constraints. Neither model's validation loss curve had fully plateaued by the final epoch, particularly the scratch model, suggesting that both models, and especially the scratch CNN, may have achieved higher accuracy with additional training epochs.

7.2 Frozen Backbone Only (No Full Fine-Tuning)

The transfer learning model was evaluated only in the feature-extraction configuration, with the entire ResNet18 backbone frozen. A full or partial fine-tuning configuration, where some or all of the backbone layers are also updated at a small learning rate, was not evaluated, and could plausibly close or widen the gap between the two approaches further.

7.3 Single Architecture per Approach

Only one from-scratch architecture and one pretrained backbone (ResNet18) were tested. The relative performance gap observed here is specific to this pair of architectures and may not generalise to other scratch architectures (e.g., deeper custom CNNs) or other pretrained backbones (e.g., VGG, MobileNet, EfficientNet).

7.4 Dataset Scope

The RVF10K dataset, while a reasonable benchmark for this project, is relatively small (10,000 images) and consists of a specific style of fake images. Performance on this dataset may not directly generalise to deepfakes generated by other, particularly newer, generative methods, or to face images captured under different conditions (lighting, pose, resolution) than those represented in this dataset.

7.5 Image-Level Classification Only

This project operates purely on static, single images. Real-world deepfake content is very often video-based, where temporal inconsistencies between frames can provide additional, often more reliable, detection signals that a single-image classifier cannot access.

8. Conclusion

This project implemented and compared two CNN-based approaches to deepfake face detection: a compact convolutional neural network trained entirely from scratch, and a ResNet18 model adapted via transfer learning with a frozen backbone. Under an identical training configuration i.e, same dataset, batch size, learning rate, optimizer, and number of epochs. The transfer learning model consistently outperformed the scratch model across every evaluation metric: 77.37% vs. 73.97% validation accuracy, 0.7736 vs. 0.7395 macro F1-score, and 0.8507 vs. 0.8196 ROC-AUC.

Notably, the transfer learning model achieved this superior performance while updating only 1,026 parameters, compared to over 6.4 million trainable parameters in the scratch model, demonstrating that the general-purpose visual features learned from a large, diverse dataset like ImageNet can be highly effective even for a specialised downstream task like deepfake detection, requiring only a small, cheaply trained classification layer on top. At the same time, this project also showed that training and inference speed are governed by a model's total computational depth, not simply by how many of its parameters are being optimised, since the transfer learning model's training time was not meaningfully faster despite its vastly smaller trainable parameter count.

Overall, this project reinforces a central lesson of the mentorship track's Week 5 and Week 6 material: for image classification tasks with limited training data, transfer learning is generally a strong default choice over training a comparable-capacity model from scratch, while acknowledging that both approaches remain competitive and that a scratch model can still achieve reasonable performance given sufficient data and training time.

9. References

1. He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. arXiv:1512.03385.

2. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR.

3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

4. Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. arXiv:1502.03167.

5. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.

6. Stanford CS231n Course Notes: Convolutional Neural Networks for Visual Recognition. https://cs231n.github.io/convolutional-networks/

7. Stanford CS231n Course Notes: Transfer Learning. https://cs231n.github.io/transfer-learning/

8. RVF10K Dataset (Real vs Fake Faces — 10K), Kaggle. https://www.kaggle.com/datasets/sachchitkunichetty/rvf10k

9. PyTorch Documentation — torchvision.models.resnet18. https://pytorch.org/vision/stable/models.html